

How much data does PolyBench move? A reuse-distance study

Symbolic data-movement growth rates and coefficients for PolyBench, under single-shot and infinitely-repeating execution, with empirical confirmation

2026-07-08

This report measures the **data-movement cost** of the PolyBench kernels and explains what drives it. We use the AutoLALA dmd analyzer, which reads a loop nest and produces *symbolic* formulas — cost as a function of the problem size, not a single number for one input. From those formulas we extract a simple, testable law for how each kernel’s data movement grows, rank the kernels by both growth rate and constant factor, and check the predictions against real cache-miss counts and wall-clock time.

Everything here is reproducible; see **Reproduce** at the end.

1. Three quantities, in plain words

When a program runs, it touches memory in some order. To reason about locality we track, for each piece of data, *how long it sits unused between two touches*. There are two natural ways to measure “how long”, and one summary number built on top of them.

Reuse interval (RI). When you touch a value and later touch it again, the reuse interval is *how many memory accesses happened in between*, counted in accesses. If you read $A[0]$, then read nine other things, then read $A[0]$ again, the reuse interval of that second $A[0]$ is 10.

Reuse distance (RD). The reuse interval counts *every* access in between; the reuse distance counts only the *distinct cache lines* touched in between. That is what actually decides whether the value is still in cache when you return to it. If those nine intervening accesses only ever hit two different cache lines, the reuse distance is 2–3 lines even though the reuse interval is 10. RD is the working-set size between two touches, in cache lines.

A cache line here is **8 doubles** (a 64-byte line holds eight 8-byte f64 values). All PolyBench arrays are double precision, so the analyzer’s block size is 8 elements. (Block size is in *elements*, not bytes.)

Data-movement distance (DMD). This is the single summary number. For every reuse in the program it adds the **square root of that reuse’s distance**:

$$\text{DMD} = \sum_{\text{reuses}} \sqrt{\text{reuse distance}}.$$

Why the square root, and why the sum? The model treats a reuse at distance d as costing $\sqrt{d}$ to service — a memory-hierarchy cost model in which data further away (a larger working

set to traverse before you get back) is more expensive, but sub-linearly so. Summing over all reuses gives one number whose *growth rate* captures how a kernel’s data traffic scales with problem size. A kernel whose reuses stay at small distance has low DMD; one that keeps reaching far back into a large working set has high DMD. We report DMD **as a formula in the problem size**, then read off its growth rate.

2. Two models of the trace: single-shot vs. infinitely repeating

A kernel’s reuse pattern depends on an assumption people rarely state: *does the kernel run once, or over and over?*

Single-shot. The kernel runs exactly once. The very last time a value is touched it is never touched again, so that touch is a **cold miss** (compulsory — the data had to be brought in at least once). This is the right model for a one-off computation.

Infinitely repeating. The kernel runs many times back-to-back — think of a linear-solver step, a stencil sweep, or an inner loop re-run every outer iteration. Now the “last” touch of a value in one pass is followed by the “first” touch of the same value in the next pass, so that value is reused — at a large distance (you sweep the whole working set in between), but reused nonetheless. This is the right model for a kernel that lives inside an outer iteration.

We implement infinite repeat the simple way: wrap the kernel in an outer loop that runs **twice**, and keep only the reuse intervals whose *second* touch lands in the second pass. That pass has a full period of history behind it, so its reuses — including the ones that wrap around the period boundary — are the steady-state ones. Two passes suffice whenever every value live in one pass is touched again in the next (true for all these kernels).

Here is the difference on a two-line example — read $A[i][j]$ and $B[j]$ inside an i, j nest. $B[j]$ is reused every i (short distance); $A[i][j]$ is touched exactly once per pass.

model	reuse distances the analyzer reports
single-shot	$2 \cdot M, 4, 2$
infinite-repeat	$2 \cdot M, 4, 2, N \cdot M + M, N \cdot M + 2, N \cdot M + 1$

Single-shot sees only B ’s short reuse and calls every $A[i][j]$ a cold miss. Infinite-repeat additionally sees that each $A[i][j]$ is re-touched next pass — at a reuse distance of about $N \cdot M$, the size of the whole array. The access *total* is identical either way ($2 \cdot N \cdot M$); infinite-repeat does not invent work, it reclassifies boundary cold misses as far-away reuses. **This is the extra information infinite-repeat gives: reuse that only exists across invocations.** We report both models throughout.

3. The growth-rate law

Write the problem size as N . Two growth rates matter:

- **access growth** a — the number of memory accesses grows like N^a (a triple loop over N does N^3 accesses, so $a = 3$).
- **reuse-distance growth** ρ — the *largest* reuse distances grow like N^ρ .

Because DMD sums $\sqrt{\text{distance}}$ over roughly N^a reuses, and the dominant reuses sit at distance $\sim N^\rho$ each contributing $\sqrt{N^\rho} = N^{\rho/2}$, the DMD grows like

$$\text{DMD} \sim \text{coeff} \cdot N^d, \quad d = a + \frac{1}{2}\rho.$$

In words: **the DMD growth exponent is the access-growth exponent plus half the reuse-distance-growth exponent.** We compute d directly and stably from the reuse-distance distribution (summing the square-root terms), which avoids the numerical cancellation you get from evaluating the fully assembled formula.

We call $d - a = \frac{1}{2}\rho$ the **headroom**. It is the most useful single number here, so plainly:

Headroom is how much *faster* a kernel’s data movement grows than its raw arithmetic. Headroom 0 means data movement grows in lockstep with work — the kernel is already as local as it can be. Large headroom means the kernel moves far more data than it computes, and a locality transformation (tiling, loop interchange) has room to recover roughly a factor of N^{headroom} .

Across the symbolic kernels ρ takes only a few clean values, so headroom clusters at a few levels — the backbone of the taxonomy in §6.

4. Same growth rate, different constant: the coefficient

Growth rate alone cannot separate two kernels that scale the same way. Two kernels can both be $\Theta(N^4)$ yet move meaningfully different amounts of data, because the **leading coefficient** in $\text{DMD} \sim \text{coeff} \cdot N^d$ differs. The coefficient is physically meaningful — it counts how many square-root-of-distance units of traffic the kernel issues per N^d — so we report it and use it to rank kernels *within* a growth class.

We extract the coefficient from the same square-root construction used for the exponent: for each dominant reuse-distance term, take the leading coefficient of its multiplicity times the square root of the leading coefficient of its distance, and sum the terms sharing the top growth order. The result is a single number coeff with $\text{DMD} \approx \text{coeff} \cdot N^d$ at large N .

The ranking is intuitive where we can check it by hand. `gemm`, `2mm`, and `3mm` all grow as N^4 (headroom 1.0), but their coefficients are 0.044, 0.088, and 0.132 — almost exactly 1 : 2 : 3, because `2mm` is two chained matmuls and `3mm` is three, each contributing one matmul’s worth of far-reaching reuse. The exponent says “these are all N^4 kernels”; the coefficient says “`3mm` moves three times the data of `gemm`.” Likewise in the flat headroom-0 band, `syr2k` (coeff 3.79) moves about $4\times$ the data of `cholesky` (0.92) at the same N^3 growth. We therefore rank kernels first by headroom, then by coefficient.

5. Results — how every kernel scales

Below are all 27 symbolic kernels the analyzer could handle, grouped by headroom. For each: the access growth a , the DMD growth d , the headroom $d - a$, and the leading coefficient (so $\text{DMD} \approx \text{coeff} \cdot N^d$). The last two columns repeat d and the coefficient under the infinite-repeat model; “—” means the doubled trace exceeded the analyzer’s counting budget (see §7). A **bold** inf-repeat order marks a kernel whose growth rate *rises* under repetition.

Headroom 1.0 — data movement grows a full factor of N faster than the work.

kernel	a	d	headroom	coeff	d (inf)	coeff (inf)
doitgen	4	5.0	1.0	0.044	5.0	0.044
jacobi-2d	3	4.0	1.0	0.468	—	—
3mm	3	4.0	1.0	0.132	4.0	0.132
2mm	3	4.0	1.0	0.088	4.0	0.088
seidel-2d	3	4.0	1.0	0.083	—	—
gemm	3	4.0	1.0	0.044	4.0	0.044
floyd-warshall	3	4.0	1.0	0.044	4.0	0.044

Headroom 0.5 — data movement grows a factor of $\sqrt{N}$ faster than the work.

kernel	a	d	headroom	coeff	d (inf)	coeff (inf)
gramschmidt	3	3.5	0.5	2.685	3.5	2.098
imperfect	3	3.5	0.5	2.568	3.5	2.656
covariance	3	3.5	0.5	0.706	3.5	0.706
correlation	3	3.5	0.5	0.706	—	—
fdtd	3	3.5	0.5	0.242	—	—
gemver	2	2.5	0.5	1.216	2.5	1.216
mvt	2	2.5	0.5	1.063	2.5	1.063
jacobi-1d	2	2.5	0.5	0.249	2.5	0.250
bicg	2	2.5	0.5	0.153	3.0	0.044
gesummv	2	2.5	0.5	0.076	3.0	0.125
atax	2	2.5	0.5	0.062	3.0	0.044

Headroom 0.0 — data movement grows in lockstep with the work; no asymptotic locality slack.

kernel	a	d	headroom	coeff	d (inf)	coeff (inf)
syr2k	3	3.0	0.0	3.789	3.0	3.827
symm	3	3.0	0.0	2.899	3.0	2.943
syrk	3	3.0	0.0	2.158	3.0	2.191
lu_decomp	3	3.0	0.0	1.899	3.0	1.881
lu	3	3.0	0.0	1.378	—	—
trmm	3	3.0	0.0	1.366	3.0	1.351
cholesky	3	3.0	0.0	0.923	3.0	0.940
trisolve	2	2.0	0.0	2.871	3.0	0.031

Two things stand out immediately. First, **every headroom value is 0.0, 0.5, or 1.0** — nothing in between. The reuse-distance growth ρ is quantized to $\{0, 1, 2\}$ across the entire suite; there is no kernel whose worst reuse distance grows like, say, $N^{1.3}$. Second, within a headroom band the **coefficients spread over 1-2 orders of magnitude**, which is exactly the ranking information the growth rate alone throws away (§4).

6. Three optimization classes

Because headroom takes only three values, the kernels fall into three groups, and the group tells you *which* locality transformation can help and *how much*.

Class A — headroom 1.0 (reuse distance grows like N^2). The dense linear-algebra and 2-D stencil kernels: gemm, 2mm, 3mm, doitgen, floyd-warshall, jacobi-2d, seidel-2d. Here the working set reached between reuses is a whole matrix, so the reuse distance grows as N^2 and the data movement outpaces the arithmetic by a full factor of N . The right transformation is **tiling**: block the loops so the reused sub-matrix fits in cache, which caps the reuse distance at the (constant) tile footprint and collapses the headroom to zero. This is the largest asymptotic win available — and the one the empirical section confirms most sharply.

Class B — headroom 0.5 (reuse distance grows like N). Matrix-vector products and mixed 1-D patterns: mvt, atax, bicg, gemver, gesummv, jacobi-1d, covariance, correlation, fdtd, gramschmidt, imperfect. The culprit is typically one array streamed “the wrong way” — read down a column while it is stored by rows, or re-streamed on each pass — so its reuse distance grows as N . The fix is **loop interchange or fusion** to bring that access’s reuse close, recovering about $\sqrt{N}$.

Class C — headroom 0.0 (reuse distance bounded). Triangular and accumulator kernels: syr, syr2k, cholesky, lu, lu_decomp, trmm, trisolve, symm. Every element’s reuse is already local — a resident accumulator, or a bounded triangular band — so there is no asymptotic locality slack. Tiling cannot change the growth rate (it may still help the constant factor). Optimization effort is better spent on vectorization and register use than on data locality.

The point of the taxonomy is predictive: given a new affine kernel, its headroom tells you up front whether a locality transformation is worth attempting at all, and if so which one and roughly what payoff to expect.

7. What infinite-repeat reveals

Comparing the two models across the whole suite gives a clean answer to “does infinite-repeat tell us anything the single-shot model doesn’t?” — **yes, for a specific and recognizable set of kernels**. Four kernels raise their DMD growth rate under repetition:

kernel	single-shot	infinite-repeat
atax	$N^{2.5}$	$N^{3.0}$
bicg	$N^{2.5}$	$N^{3.0}$
gesummv	$N^{2.5}$	$N^{3.0}$
trisolve	$N^{2.0}$	$N^{3.0}$

These are exactly the kernels that **read their matrix once per invocation**. In a single pass, each matrix element is touched once and then never again, so the single-shot model files it as a cold miss and it contributes nothing to the reuse-driven part of DMD. But if the kernel repeats — atax inside a Krylov solver, trisolve inside a Newton iteration — the matrix is read again on the next pass, a genuine reuse at distance $\approx N^2$ (the matrix footprint). That adds a term of size (number of matrix elements $\sim N^2$) $\times \sqrt{N^2} = N^3$, lifting the DMD order. Infinite-repeat surfaces this cross-invocation reuse; single-shot cannot see it.

The contrast within Class B is telling. mvt and gemver are also matrix-vector kernels, yet they do **not** jump — because they already touch the matrix more than once per pass (mvt multiplies by A and $A^\top$; gemver applies rank-1 updates), so their matrix reuse is captured within a single pass and repetition adds nothing new to the leading term. So the jump is a real signal: it distinguishes “the matrix is streamed once and only reused across invocations” from “the matrix is reused within each invocation.”

Practically: if you are optimizing one of the jumping kernels *inside an outer iteration*, the analysis says the matrix reuse across iterations is worth capturing (keep the matrix resident if it fits), whereas the single-shot view would have told you the matrix traffic is unavoidable cold-miss volume.

Where infinite-repeat did not complete (jacobi-2d, seidel-2d, correlation, fdtd, lu — the “—” rows in §5), the doubled trace pushed the Barvinok counter past its operation budget. For every kernel where it did complete, the headroom stayed the same or rose — never fell — consistent with the model only ever *adding* boundary reuses, never removing intra-pass ones.

8. Empirical confirmation

The growth-rate law is a claim about reuse distances, and reuse distance is what decides cache behavior. So we check the predictions on real hardware with the actual C kernels (not the model): **cache-miss scaling** with cachegrind, and **wall-clock time per access** over a size sweep. We take one kernel from each class. The prediction in each case is about the *last-level* miss rate, because that is what a growing reuse distance eventually blows.

Class A — matmul (predicted headroom 1: reuse distance $\sim N^2$). As N grows, the reused operand (a full $N \times N$ matrix) eventually overflows last-level cache. The last-level miss rate should stay tiny while it fits, then jump once it doesn’t — and tiling, which caps the reuse distance at the tile size, should prevent the jump.

last-level miss rate	N=128	N=256	N=384	N=512
naive	0.18%	0.08%	0.07%	3.55%
tiled	0.12%	0.05%	0.04%	0.16%

The naive kernel’s miss rate is flat and small up to $N=384$, then **jumps about $50\times$** at $N=512$ — exactly when the working set crosses the simulated 2 MB cache. Tiling holds it at 0.16%, **$22\times$ lower** than naive at $N=512$ and still flat. Wall-clock time per access rises with the miss rate — 1.67, 2.01, 2.15 ns at $N = 256, 512, 1024$ — then saturates at memory latency. This is the headroom-1 signature: an unbounded reuse distance that tiling removes.

Class B — mvt (predicted headroom 0.5: reuse distance $\sim N$). One array is streamed transposed; its reuse distance grows as N , so the miss rate should climb with N until that access no longer fits, and loop interchange should flatten it.

last-level miss rate	N=512	N=1024	N=2048
naive	9.9%	31.0%	31.2%
interchanged	7.4%	7.5%	7.5%

The naive miss rate **grows** from 9.9% to 31% as the transposed access outgrows cache; loop interchange fixes the access order and pins it at 7.5% across all sizes — about **$4\times$ lower** at $N=2048$. Headroom- $\frac{1}{2}$ signature: a growing reuse distance from one mis-ordered access, capped by interchange.

Class C — syrk (predicted headroom 0: reuse distance bounded). Every reuse is already local, so the miss rate should be flat in N with nothing to fix.

last-level miss rate	N=128	N=256	N=384	N=512
naive	0.31%	0.13%	0.12%	0.09%

It is flat — if anything *declining*, as one-time costs amortize — and wall-clock time per access falls slightly ($0.51 \rightarrow 0.39 \rightarrow 0.38$ ns) rather than rising. There is no locality problem to solve. Headroom-0 signature.

All three predicted behaviors are observed, on real cache hardware, for the class the growth-rate analysis assigned each kernel.

A note on ground truth. Cachegrind is used only as an *independent check of the reuse-distance prediction* — does the miss rate move the way the model says? — not as a performance oracle. Wall-clock time is the performance measure. (PMU counters via perf were unavailable on this host, which runs with perf_event_paranoid=4; we did not weaken that host setting, and cachegrind plus timing were sufficient.)

9. Finite ranges: the local exponent, and what doubling N actually costs

Everything up to here is asymptotic. In practice you have a size *range* and you ask: “if I double N from here, what does it cost?” The order d answers only the limit — it says DMD scales by 2^d per doubling. Over a real octave $[N, 2N]$ the honest quantity is the **local exponent**, the log-log slope measured right at your size,

$$p(N) = \frac{\log(\text{DMD}(2N)/\text{DMD}(N))}{\log 2},$$

computed exactly from the symbolic formula (local_analysis.py, with $\text{DMD}(N)$ evaluated as $\sum \text{multiplicity} \cdot \sqrt{\text{distance}}$ over the reuse-distance bins — no cancellation). Here is gemm (Class A, order $d = 4$):

N	$\text{DMD}(2N)/\text{DMD}(N)$	local $p(N)$	naive 2^d	access slope	local headroom
64	11.14	3.48	16.0	2.99	0.48
256	13.53	3.76	16.0	3.00	0.76
512	14.47	3.85	16.0	3.00	0.86
1024	15.09	3.92	16.0	3.00	0.92
4096	15.70	3.97	16.0	3.00	0.97

Two things to read off. First, the naive 2^d **overestimates** the real cost of doubling at practical sizes: at $N=64$, doubling multiplies DMD by $11.1\times$, not the asymptotic $16\times$; the local exponent 3.48 is the honest growth rate there, and it only creeps toward 4.0. Second, the **local headroom** (last column) starts at 0.48 and climbs to 0.97 — so at small N gemm’s per-access data movement grows far less than the asymptotic headroom of 1.0 promises. You are *pre-asymptotic*: the tiling payoff is latent, and how latent depends on where your N sits.

Contrast a headroom-0 kernel, syrk:

N	$\text{DMD}(2N)/\text{DMD}(N)$	local $p(N)$	naive 2^d	local headroom
64	7.97	2.99	8.0	0.011
256	7.99	3.00	8.0	0.003
4096	8.00	3.00	8.0	0.000

`syrrk` sits *at* its asymptote immediately — local exponent 3.00, local headroom 0.00 across the whole range. There is no pre-asymptotic regime and no latent payoff, because there is no growing reuse distance to wait for. This is the finite- N face of “headroom 0 means nothing to optimize.”

The cliff the smooth exponent hides. DMD is a smooth polynomial, so its local exponent drifts continuously. The real *miss* curve does not — it has a cliff at the size where the dominant reuse distance fills the cache. `gemm`’s dominant reuse-distance term is $0.125 N^2$ cache lines, i.e. $0.125 N^2 \times 64 \text{ B} = 8N^2$ bytes, which fills a 2 MB last-level cache at

$$N^* = \sqrt{2 \text{ MB} / 8 \text{ B}} = 512.$$

The cachegrind sweep in §8 put `gemm`’s last-level miss-rate jump at **exactly** $N=512$. The model’s reuse-distance *coefficient* predicted the measured cliff location, not merely its existence. `syrrk`’s dominant growing distance, by contrast, is only $\sim N^1$ and carries a vanishing fraction of the accesses, so its crossing size is astronomically large ($N^* \approx 2.6 \times 10^5$) and no cliff appears in range — consistent with its flat measured miss rate.

So the complete finite-range recipe is: use the **local exponent** $p(N)$ (exact, from the formula) for the smooth cost, and separately test whether your octave $[N, 2N]$ **straddles** $N^* = \sqrt{C/c}$ for the cliff. The derivative refines the exponent; the threshold catches the discontinuity the exponent cannot see.

10. What we could and could not analyze

Of the 53 PolyBench programs (31 symbolic-size, 22 fixed-size), the analyzer handled **48 under the single-shot model**, and **43 of those also under infinite-repeat**. That single-shot figure is up from 41 before this work: the reduction-loop fix in §11 recovered the six accumulator kernels (convolution, `symm`, `gramschmidt` in both size families) that the extractor had previously refused. The growth-rate table in §5 covers the 27 symbolic kernels that yield a size-dependent formula (the fixed-size kernels give one number, not a growth rate, and are used only as numeric cross-checks).

The five programs we could not analyze fall into three honest buckets:

- **Not representable** — `adi`, `deriche`, `durbin`. These use genuinely non-affine array subscripts (an index computed from loaded data) or MLIR syntax the current toolchain no longer parses. They are outside the affine model, and we let them error rather than silently drop accesses. (None were in the legacy AutoLALA reference set either.)
- **Over the counting budget** — `heat-3d`. A deep, heavily-parametric 3-D stencil whose reuse relation exceeds the Barvinok operation budget even single-shot.
- **Excluded** — `fdtd-apml`, a known pathological case for the counter (it times out); dropped by request.

Infinite-repeat additionally could not complete on the five heaviest kernels that *do* analyze single-shot (`jacobi-2d`, `seidel-2d`, `correlation`, `fdtd`, `lu`): doubling the trace doubles the outer schedule dimension and pushes the polytope past the counting budget. One further

kernel, convolution, analyzes but its symbolic bounds ($N-K$ in both loop dimensions) leave the access-count formula without a clean single growth order, so it is omitted from the ranked table.

None of these gaps affect the central findings, which rest on the 27 symbolic kernels that do produce clean formulas.

11. Implementation notes

Two toolchain changes made this study possible; both live in this branch.

Loading kernels with reductions. The MLIR extractor had been rejecting any loop that carried a value across iterations (an `iter_args` reduction), which blocked every kernel with an inner accumulator (convolution, `symm`, `gramschmidt`). The DMD model only tracks memory movement, so a value carried in a register is irrelevant — the legacy AutoLALA extractor simply ignored such carried values. We restored that: an `affine.for` with `iter_args` is treated as an ordinary loop, its carried scalar and `affine.yield` ignored, its inner loads/stores flowing through unchanged. Scratch (de)allocations (`memref.alloc/alloca/dealloc`) are likewise ignored. This recovered the six reduction kernels. Kernels with genuinely non-affine subscripts (`adi`, `durbin`, `deriche`) remain out of scope — not representable — and we let them error rather than silently drop accesses.

The infinite-repeat model. We added `--infinite-repeat` to `dmd-cli`. It wraps the program in a two-iteration outer loop and keeps only the reuse intervals whose consuming access lands in the second iteration, as in §2. This is simpler than the original AutoLALA scheme (no symbolic repeat count, no normalization): two concrete passes suffice and the second-pass filter yields the steady-state distribution directly. A unit test pins the behavior to the §2 example — the wraparound reuse must appear at footprint distance and the access total must stay at one period.

Block size. The analyzer’s block size is in elements. A 64-byte cache line is 8 doubles, so we pass `--block-size 8` (16 would model single precision). This matches the line cachegrind simulates.

Reproduce

```
python3 run_analysis.py both --resume      # analyze all kernels, both models -> results/
python3 analyze_math.py                   # growth rates + coefficients -> order_table.json
python3 local_analysis.py                  # local exponent / doubling cost / cache threshold N*
cd confirm && python3 sweep.py              # cachegrind miss-scaling -> cg.json
pandoc REPORT.md -o REPORT.pdf --pdf-engine=xelatex \
-V mainfont="DejaVu Serif" -V monofont="DejaVu Sans Mono"
```